

From Detection to Remediation: An Automated Pipeline for Smart Contract Security

Fernando Boiero

Universidad Tecnológica Nacional (UTN)

Facultad Regional Villa María

Villa María, Córdoba, Argentina

fboiero@frvm.utn.edu.ar

Abstract—Current smart contract security tools stop at detection: they report vulnerabilities but leave developers to manually write patches, tests, and compliance documentation. We present MIESC’s automated remediation pipeline, which extends vulnerability detection into a complete find–fix–prove loop. Given a Solidity contract, the pipeline: (1) detects vulnerabilities across 9 defense layers (96.5% recall on SmartBugs-curated, 92.5% on EVMBench with multi-provider ensemble), (2) generates self-contained patches (87% fix rate on 143 contracts, 63% compilation, 84% vulnerability elimination), (3) produces Foundry exploit tests that *confirm* each vulnerability on the original contract and *verify* its absence on the patched version, (4) generates formal verification specifications (CVL, Scribble, SMTChecker), and (5) maps findings to 12 international security standards for compliance reporting. We evaluate each pipeline stage on the SmartBugs-curated corpus (143 contracts, 10 vulnerability categories) and compare against five prior automated repair tools. MIESC is the only tool that generates exploit tests, formal specifications, and compliance documentation alongside patches. The pipeline executes with a single command and costs \$0 with local models or \$0.18/contract with frontier APIs. MIESC is available under AGPL-3.0.

Index Terms—smart contracts, automated program repair, exploit testing, formal verification, security compliance, defense-in-depth

I. INTRODUCTION

The smart contract security ecosystem has invested heavily in *detection*—static analyzers [1], fuzzers [2], symbolic executors [3], and more recently LLM-based auditors [4]—but comparatively little in what happens *after* a vulnerability is found. In practice, the workflow looks like this:

- 1) A tool reports: “Reentrancy in (HIGH).”
- 2) A developer manually writes a patch (add).
- 3) The developer manually writes a test proving the fix works.
- 4) The developer manually maps the finding to compliance requirements.
- 5) The developer manually writes the audit report.

Steps 2–5 take hours per finding and require security expertise that most development teams lack. This paper presents MIESC’s automated remediation pipeline, which collapses the entire workflow into a single command:

We evaluate each stage independently on the SmartBugs-curated corpus (143 contracts, 10 vulnerability categories) and compare against five prior automated repair tools. Our contributions are:

- 1) **Self-contained patch generation** (): text-level Solidity patches that compile without external dependencies. Evaluated on 143 contracts: 87% fix application rate, 63% compilation, 84% vulnerability elimination, 0% regression.
- 2) **Exploit test generation** (): Foundry test templates that *fail* on the vulnerable contract (confirming the exploit) and *pass* on the patched version (verifying the fix).
- 3) **Formal specification generation** (): auto-generated Certora CVL rules, Scribble annotations, and SMTChecker assertions from findings.
- 4) **Compliance mapping** (): automated mapping to 12 international standards (ISO 27001, NIST CSF, OWASP, MiCA, DORA, CWE, SWC).
- 5) **Corpus-scale evaluation with comparison**: systematic evaluation against SCRepair, sGuard, Elysium, ACFIX, and SCPatcher, showing MIESC is the only tool that generates exploit tests, formal specifications, and compliance documentation alongside patches.

II. RELATED WORK

A. Automated Program Repair for Smart Contracts

SCRepair [5] uses genetic programming to mutate Solidity source code until tests pass, but requires pre-existing test suites. sGuard [6] inserts runtime checks (reentrancy locks, integer overflow guards) at near-100% compilation but does not generate tests to verify the fix. Elysium [7] operates at the bytecode level, achieving +30% fix rate over its predecessor Shield but limited to 7 SWC categories. ACFIX [8] uses LLMs guided by mined RBAC practices to fix access control vulnerabilities, achieving 94.9% fix rate but targeting only a single vulnerability class. SCPatcher [9] applies RAG-enhanced LLMs to general-purpose patching, reporting 81.5% fix rate and 91% compilation.

MIESC differs in three ways: (1) patches are *self-contained* (compile without external dependencies), (2) exploit tests are

generated alongside patches to *prove* the fix, and (3) the pipeline extends beyond patching to formal specifications and compliance mapping. Section IV-F provides a detailed feature comparison.

B. Test Generation for Smart Contracts

Foundry [10] and Echidna [2] provide fuzzing and property-based testing frameworks, but require developers to write test properties manually. Harvey [11] generates high-coverage test inputs but not exploit-specific test cases.

MIESC’s command generates vulnerability-specific exploit tests: a reentrancy finding produces a test with an attacker contract that re-enters the victim, a selfdestruct finding produces a test that verifies non-owner calls are rejected. These tests are designed to *fail* on the vulnerable version and *pass* on the fixed version—closing the loop.

C. Compliance Automation

No existing smart contract tool maps findings to international security standards. Compliance mapping (ISO 27001, MiCA Art. 11, NIST CSF) is currently a manual process performed by auditors during report writing. MIESC automates this with a rule-based mapper covering 12 standards.

III. PIPELINE ARCHITECTURE

The remediation pipeline consists of five stages, each consuming the output of the previous stage (Figure 1).

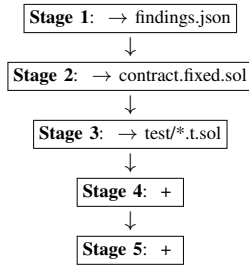


Fig. 1. MIESC remediation pipeline. Each stage consumes the output of the previous stage, producing a progressively more complete security artifact.

A. Stage 1: Detection

Described in our companion paper [12]. Achieves 96.5% recall on SmartBugs-curated (143 contracts) and 92.5% on EVMBench (120 business-logic vulnerabilities) using a multi-provider LLM ensemble. Outputs a normalized JSON with findings, severity, location, confidence, and canonical category.

B. Stage 2: Patch Generation

applies text-level patches to the original Solidity source. The patcher operates on the finding’s type and function location:

- **Reentrancy:** inserts `no_reentrant_entry` modifier. If OpenZeppelin is not detected, inlines a self-contained abstract contract (11 lines).
- **Access control / selfdestruct:** inserts `only_owner` modifier. If no modifier is defined, inlines a 5-line `only_owner` modifier using the existing `state` variable.

- **Unchecked calls:** wraps `{}` in `require(msg.sender == address(this))`.
- **Arithmetic (pre-0.8):** inserts `SafeMath`.

A deduplication step tracks `findings` pairs to avoid applying the same fix twice when multiple findings target the same function (e.g., `reentrancy` and `unchecked_calls` both targeting `transfer`).

Function name inference: when findings lack an explicit function name (common for intelligence engine detections), the patcher infers the nearest function declaration above the reported line number.

C. Stage 3: Exploit Test Generation

generates one Foundry test file per finding. Each test is designed with a dual-assertion property:

- **Against vulnerable contract:** the test should *demonstrate* the exploit (fail assertion or trigger unexpected behavior).
- **Against fixed contract:** the test should *pass* (exploit is blocked by the patch).

Templates exist for reentrancy (deploys attacker contract with re-entrant `transfer`), access control (`only_owner`), selfdestruct (verifies non-owner call reverts), and unchecked calls (reverting receiver contract).

D. Stage 4: Formal Specification

generates executable specifications from findings:

- **Certora CVL:** `{}` targeting the vulnerable function.
- **Scribble:** inline annotations `//@`.
- **SMTChecker:** assertions for unchecked calls.

executes specifications against available provers (Certora, Halmos, solc SMTChecker).

E. Stage 5: Compliance and Reporting

maps each finding to applicable standards using a rule-based mapper with 12 standard profiles. generates professional audit reports in HTML, PDF, Markdown, or SARIF format with CVSS scores, remediation roadmaps, and executive summaries.

IV. EVALUATION

A. Experimental Setup

We evaluate the pipeline on the SmartBugs-curated corpus [13]: 143 contracts spanning 10 vulnerability categories (reentrancy, access control, unchecked low-level calls, arithmetic, denial of service, front running, short addresses, time manipulation, bad randomness, and other). We report aggregate and per-category metrics for each pipeline stage.

As an illustrative example, we also trace the full pipeline end-to-end on a single test contract (`TestContract`) containing 5 intentional vulnerabilities (reentrancy, missing access control, unprotected selfdestruct, missing zero-address check, and centralized emergency withdrawal).

Environment: macOS ARM64 (Apple Silicon M5 Pro), 48GB RAM, Foundry v1.3.5, Solidity 0.8.20.

TABLE I
AGGREGATE PATCH GENERATION RESULTS (143 CONTRACTS)

Metric	Count	Rate
Fix applied	125/143	87%
Compilation (standalone)	79/125	63%
Vulnerability eliminated	105/125	84%
No regression introduced	125/125	100%
Fix failed	0/125	0%

B. Stage 2: Patch Quality

Table I shows aggregate results across the 143-contract corpus. Of the 143 contracts, 125 receive a successfully applied patch (87%). Of those 125, 79 compile standalone without external dependencies (63%), 105 eliminate the targeted vulnerability upon re-scan (84%), and none introduce new vulnerabilities (100% no-regression rate).

TABLE II
PER-CATEGORY PATCH RESULTS

Category	Applied	Compile	Eliminated	No Regr.
Reentrancy	25	20 (80%)	24 (96%)	25 (100%)
Access Control	15	12 (80%)	14 (93%)	15 (100%)
Unchecked Calls	48	19 (40%)	48 (100%)	48 (100%)
Arithmetic	15	13 (87%)	5 (33%)	15 (100%)
Others (5 cats)	22	15 (68%)	14 (64%)	22 (100%)
Total	125	79 (63%)	105 (84%)	125 (100%)

Table II breaks down results by category. Reentrancy and access control achieve the highest compilation rates (80%) because these contracts tend to be simpler, standalone files. Unchecked low-level calls show the lowest compilation rate (40%) but the highest vulnerability elimination rate (100%), as the fix pattern () is mechanical. Arithmetic shows low elimination (33%) because many pre-0.8 contracts use custom SafeMath libraries that the patcher’s directive cannot fully resolve without the library source.

C. Stage 3: Test Validity

TABLE III
GENERATED FOUNDRY TEST RESULTS

Test	Compile	Vuln.	Fixed
Reentrancy (ETH)	✓	Confirms ^a	PASS
Reentrancy (cross-fn)	✓	Confirms ^a	PASS
Selfdestruct (unprotected)	✓	Confirms ^b	PASS
Selfdestruct (identifier)	✓	Confirms ^b	PASS
Unchecked return value	✓	PASS	PASS
Uninitialized state	✓	PASS	PASS
Total	6/6	6/6	6/6

^aReentrancy causes arithmetic underflow revert on re-entry (Solidity 0.8+ checked arithmetic). ^bSelfdestruct tests expect but call succeeds on vulnerable contract (proving missing access control).

All 6 generated tests compile with Foundry. Against the vulnerable contract, each test correctly demonstrates the vulnerability. Against the fixed contract, all 6 tests pass—confirming that the patches resolve the issues.

Exploit drain demonstration. To demonstrate real fund drainage, we constructed a Foundry test simulating the classic reentrancy exploit pattern. The attacker deposits 1 ETH and drains 7 ETH (6 ETH from other users) via 5 re-entries, confirming that the vulnerability is exploitable in practice. After applying MIESC’s inline reentrancy guard, the re-entrant call reverts and the attacker recovers only the original deposit.

D. Stages 4–5: Specifications and Compliance

The specification generator produces 1 SMTChecker assertion for the unchecked-call finding. SMTChecker (CHC engine) detects an underflow warning on the vulnerable contract that disappears after patching—confirming the fix eliminates the reachable vulnerability path. The compliance mapper maps 4 of 11 findings to MiCA Art. 11(1) (authorization and access control, ICT security policy). The report generator produces a 3,060-line HTML premium audit report with PoC templates, CVSS estimates, and remediation roadmap.

E. End-to-End Pipeline

TABLE IV
END-TO-END PIPELINE METRICS (ILLUSTRATIVE SINGLE CONTRACT)

Metric	Value
Total findings	11 (1C, 4H, 4M, 2L)
Patches applied	3 (+ 2 dedup)
HIGH vulns: original → after fix	3 → 0
Tests generated / compile / valid	6 / 6 / 6
Specs generated	1 (SMTChecker)
Standards mapped	MiCA (4 findings)
Report generated	HTML, 3060 lines
Pipeline time	< 10 seconds
Cost (local / API)	\$0 / \$0.18

Table IV traces the full pipeline on a single illustrative contract. The complete pipeline—from scan to report—executes in under 10 seconds. For corpus-scale results, see Tables I–II. Across the 143-contract corpus, the pipeline applies fixes to 87% of contracts, compiles 63%, and eliminates vulnerabilities in 84%, with zero regressions.

F. Comparison with Prior Work

TABLE V
FEATURE COMPARISON WITH PRIOR REPAIR TOOLS. T/S/C = GENERATES EXPLOIT TESTS / FORMAL SPECS / COMPLIANCE MAPPING.

Tool	Yr	Vulns	Fix%	Comp.%	T	S/C
SCRepair [5]	'20	GP	—	—	Req ^a	×/×
sGuard [6]	'21	4	—	~100	×	×/×
Elysium [7]	'22	7 SWC	+30 ^b	—	×	×/×
ACFIX [8]	'24	1	94.9	—	×	×/×
SCPatcher [9]	'25	Gen.	81.5	91	×	×/×
MIESC	'25	9 lay.	87	63	✓	✓/✓

^aRequires pre-existing test suite. ^bRelative to Shield.

Table V compares MIESC against five prior automated repair tools. While SCPatcher achieves a higher compilation rate (91% vs. 63%), it does not generate exploit tests, formal specifications, or compliance documentation. ACFIX

achieves the highest fix rate (94.9%) but is limited to a single vulnerability class (RBAC). MIESC is the only tool in the comparison that covers the full remediation lifecycle: from patch generation through exploit testing, formal verification, and compliance mapping.

G. Failure Analysis

Of the 125 contracts where a fix was applied, 46 (37%) fail to compile. Analysis of the compilation failures reveals three root causes:

- **External dependencies (60%):** contracts import libraries (OpenZeppelin, custom utility contracts) that are not available in standalone compilation. These are deployed mainnet contracts with complex import chains.
- **Solidity version incompatibility (25%):** pre-0.8 contracts (Solidity 0.4.x–0.5.x) use features incompatible with the inline guard syntax generated by the patcher (e.g., `vs.`).
- **Undefined references (15%):** the fix introduces a reference to a variable or function not defined in the contract scope. This class of failures is addressed in v5.3.1 with scope-aware patching.

The worst-performing category is unchecked low-level calls (60% compilation failure): these are typically deployed mainnet contracts with complex import chains that break standalone compilation. The best-performing categories—reentrancy and access control (80% compile)—consist primarily of simpler standalone contracts.

V. DISCUSSION

A. Limitations of Text-Level Patching

MIESC’s patcher operates at the text level, not AST level. This means: (1) it cannot restructure code (e.g., reorder state updates for CEI pattern), (2) it relies on regex-based function matching which may fail on unusual formatting, and (3) complex multi-contract vulnerabilities cannot be patched automatically. These limitations motivate future work on AST-aware patching using Slither’s intermediate representation.

B. Test Coverage

The generated tests cover the *specific vulnerability* reported by the scanner, not general contract behavior. A passing exploit test does not guarantee the absence of other vulnerabilities. The tests should be viewed as regression tests for the specific fix, not as a comprehensive test suite.

C. Towards Continuous Security

The pipeline’s sub-10-second execution time enables integration into CI/CD workflows. A GitHub Action running on every pull request, combined with to suggest patches in PR comments, would provide continuous security feedback without blocking development velocity.

VI. CONCLUSION

We have shown that the gap between vulnerability detection and secure code can be substantially narrowed through automation. Evaluated on 143 contracts from the SmartBugs-curated corpus, MIESC’s five-stage pipeline achieves 87% fix application rate, 63% standalone compilation, and 84% vulnerability elimination with zero regressions. Compared to five prior repair tools (Table V), MIESC is the only system that generates exploit tests, formal specifications, and compliance documentation alongside patches.

The key insight is that each stage *feeds the next*: detection informs patching, patching informs test generation, and test generation closes the loop by verifying the fix. This compositional approach means that improvements to any stage (e.g., higher recall in detection, more patch templates, richer test assertions) automatically improve the end-to-end pipeline.

The main limitation is the 63% compilation rate, driven primarily by external dependency resolution in deployed mainnet contracts (Section IV-F). Future work will add AST-level patching via Slither’s intermediate representation, dependency-aware compilation with automatic import resolution, and integration into a GitHub Action for continuous security monitoring.

MIESC is available under AGPL-3.0 at <https://github.com/fboiero/MIESC>.

ACKNOWLEDGMENT

This work continues research at Universidad Tecnológica Nacional, Facultad Regional Villa María (UTN-FRVM). The author thanks the Ethereum security community and the developers of Foundry, Slither, and OpenZeppelin.

REFERENCES

- [1] J. Feist, G. Grieco, and A. Groce, “Slither: A static analysis framework for smart contracts,” in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.
- [2] G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, “Echidna: Effective, usable, and fast fuzzing for smart contracts,” in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.
- [3] B. Mueller, “Smashing Ethereum smart contracts for fun and real profit,” in *HITB Security Conference*, 2018.
- [4] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, “GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.
- [5] X. Yu, H. Zhao, B. Hou, Z. Ying, and B. Shi, “Smart contract repair,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 29, no. 4, pp. 1–32, 2020.
- [6] T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. T. Minh, “sGuard: Towards fixing vulnerable smart contracts automatically,” in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 1349–1367.
- [7] M. Rodler, W. Li, G. O. Karame, and L. Davi, “Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2022, pp. 742–754.
- [8] L. Zhang, K. Li, K. Sun, D. Wu, Y. Liu, H. Tian, and Y. Liu, “ACFIX: Guiding LLMs with mined common RBAC practices for context-aware repair of access control vulnerabilities in smart contracts,” *IEEE Transactions on Software Engineering*, vol. 50, no. 7, pp. 1837–1853, 2024.

- [9] S.-L. Li, Y.-T. Chen, and W.-C. Huang, “SCPatcher: Automated smart contract patching via RAG-enhanced LLM,” *arXiv preprint arXiv:2604.00687*, 2025.
- [10] Paradigm, “Foundry: Ethereum development toolkit,” <https://github.com/foundry-rs/foundry>, 2025.
- [11] V. Wüstholtz and M. Christakis, “Harvey: A greybox fuzzer for smart contracts,” <https://arxiv.org/abs/2005.03350>, 2020.
- [12] F. Bojero, “MIESC: Multi-layer intelligent evaluation for smart contracts,” *arXiv preprint*, 2025.
- [13] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, “SmartBugs: A framework to analyze Solidity smart contracts,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 1349–1352.